Plume PR 11

Security Review

Cantina Managed review by:
R0bert, Lead Security Researcher
Slowfi, Security Researcher

January 7, 2026

Contents

1 Introduction **2**
1.1 About Cantina . . . 2
1.2 Disclaimer . . . 2
1.3 Risk assessment . . . 2
1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
2.1 Scope . . . 3

3 Findings **4**
3.1 Medium Risk . . . 4
3.1.1 `quoteSend` redeem path reverts on `NestVault` . . . 4
3.1.2 Hardcoded zero `maxFee` in CCTP burn requests . . . 4
3.1.3 Stuck funds on invalid hook data in `relay` flow . . . 5
3.2 Low Risk . . . 5
3.2.1 Refund reuses untrusted finality threshold . . . 5
3.2.2 QuoteRelay lacks composer/selector validation . . . 6
3.2.3 Missing validation for unmapped destination EIDs in `eidToDomain` . . . 6
3.2.4 Incorrect amount used in `quoteRelay` fee estimation . . . 7
3.3 Informational . . . 7
3.3.1 Fast transfer fee breaks CCTP deposit hooks . . . 7
3.3.2 Standard fee switch bricks burns/refunds . . . 8
3.3.3 Open `destinationCaller` lets others consume CCTP message . . . 9
3.3.4 Unused import . . . 9
3.3.5 Payable `send` function does not enforce `msg.value == 0` . . . 9

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Plume is a public, EVM-compatible blockchain built for the next evolution of Real World Assets (RWAs).

From Dec 2nd to Dec 4th the Cantina team conducted a review of nest-contracts on commit hash 1a63e711. The team identified a total of **12** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	3	3	0
Low Risk	4	3	1
Gas Optimizations	0	0	0
Informational	5	3	2
Total	**12**	**9**	**3**

2.1 Scope

The security review had the following components in scope for nest-contracts on commit hash 1a63e711:

```
contracts
├── cctp
│   ├── BaseCCTPRelayer.sol
│   └── NestCCTPRelayer.sol
└── ovault
    └── NestVaultComposer.sol
```

(Files changed in PR 11)

3 Findings

3.1 Medium Risk

3.1.1 `quoteSend` redeem path reverts on `NestVault`

Severity: Medium Risk

Context: VaultComposerSyncUpgradeable.sol#L331-L364

Description: `VaultComposerSyncUpgradeable.quoteSend` uses `vault.previewRedeem` when quoting an asset send (shares in → assets out). `NestVaultCore.previewRedeem` always reverts with `ERC7540_ASYNC_FLOW`, so any redeem-side `quoteSend` against `NestVault` fails outright. That blocks callers from estimating fees/amounts for redeem-and-send flows (e.g., compose redeem or auth-only `redeemAndSend`), and even a hypothetical preview would not match the actual `instantRedeem` fee logic used in `NestVaultComposer._redeem`. As a result, users/relayers cannot rely on `quoteSend` for redemption routes and may under/over-fund `msg.value` or misstate slippage.

```
// VaultComposerSyncUpgradeable.quoteSend (redeem branch)
if (_targetOFT == ASSET_OFT()) {
    uint256 maxRedeem = vault.maxRedeem(_from);
    if (_vaultInAmount > maxRedeem) revert /*...*/;
    _sendParam.amountLD = vault.previewRedeem(_vaultInAmount); // NestVaultCore reverts
    ↪  here
}
```

```
// NestVaultCore.previewRedeem
function previewRedeem(uint256) public pure override returns (uint256) {
    revert Errors.ERC7540_ASYNC_FLOW();
}
```

Recommendation: Override `quoteSend` in `NestVaultComposer` to avoid calling `previewRedeem` and instead simulate `instantRedeem` (including fees) or otherwise return the expected asset amount for share burns. Ensure the quote reflects actual redemption math so callers can fund `msg.value` and slippage correctly; keep the deposit/share branch unchanged.

Plume Network: Fixed in commits 5ca6e275 and 2d358aa2.

Cantina Managed: Fix verified.

3.1.2 Hardcoded zero `maxFee` in CCTP burn requests

Severity: Medium Risk

Context: NestCCTPRelayer.sol#L342

Description: The function `_depositForBurn` from contract `NestCCTPRelayer` sets the `maxFee` field of the CCTP burn request to `0` and the associated `quoteSend` logic assumes a zero-fee environment. This design relies on the current behaviour of CCTP chains where `getMinFeeAmount` returns zero and only the standard transfer mode is effectively used.

However, the CCTP `TokenMessengerV2` contract enforces that `maxFee` must be greater than or equal to the value returned by `getMinFeeAmount(amount)`. If any supported CCTP domain configures a non-zero minimum fee, or if a transfer mode with non-zero `minFeeAmount` is used, calls that set `maxFee = 0` will revert when attempting to process the burn or a refund. This tightly couples the protocol's behaviour to current CCTP fee settings and can cause cross-chain operations or refunds to fail once `minFeeAmount` is updated on Circle's side.

Additionally, returning a zero fee in `quoteSend` is misleading in scenarios where a non-zero minimum fee applies, as it prevents callers from correctly estimating the total cost of the operation.

Recommendation: Consider to compute `maxFee` in the function `_depositForBurn` from contract `NestCCTPRelayer` based on the current CCTP fee configuration, for example by querying `TokenMessengerV2.getMinFeeAmount(amount)` and ensuring that `maxFee` is at least this value. Consider to expose configuration (such as maximum fee basis points and transfer speed selection) on `NestCCTPRelayer` so that integrators can choose the appropriate CCTP transfer mode while maintaining

`maxFee >= minFeeAmount`. Additionally, consider to update `quoteSend` to return a fee value consistent with the chosen CCTP fee configuration, so callers receive accurate information about the expected cost.

Plume Network: The initial assumption was that we'd use only the standard transfer. However, although the `minimumFee` is zero for all CCTP chains and all chains apart from Sei do not support the standard transfer fee switch according to the docs (https://developers.circle.com/cctp/technical-guide#standard-transfer-fee-switch), we must check `TokenMessengerV2.getMinFeeAmount(amount)` and allow users to choose the transfer speed. Added max fee basis points configuration and related error handling in NestCCTPRelayer. Fixed in commit c2742dbe.

Cantina Managed: Fix verified.

3.1.3 Stuck funds on invalid hook data in `relay` flow

Severity: Medium Risk

Context: BaseCCTPRelayer.sol#L128-L141

Description: The function `relay` from contract `BaseCCTPRelayer` first calls `_receiveMessage` to validate the CCTP message, mint USDC, and record the received amount in `nonceAmountReceived[nonce]`. It then calls `_executeHook` to process the hook associated with the message.

In `NestCCTPRelayer`, the hook is validated by `_validateHookData`, where multiple conditions can cause the hook to be considered invalid (for example, insufficient length, unexpected composer address, unexpected function selector, or `amount` greater than `nonceAmountReceived[nonce]`). When the hook is invalid, `_executeHook` returns `(_hookSuccess = false, _hookReturnData = errorData)` without writing anything to `retryHookQueue` for the corresponding nonce.

Both `retryHook` and `refund` from contract `BaseCCTPRelayer` require that `retryHookQueue[nonce]` is set for the message; otherwise, they revert with `MessageNotFound`. As a consequence, when the hook data is invalid, USDC is minted to the relayer and `nonceAmountReceived[nonce]` is populated, but the message cannot be retried nor refunded through the intended public interfaces. The only way to move these funds is via the privileged `recoverToken` functionality, which loses the per-nonce accounting context and diverges from the expected retry/refund semantics.

Recommendation: Consider to ensure that messages with invalid hook data can still follow a well-defined refund path instead of leaving the funds effectively stuck under normal flows. For example, consider to:

- Update `_executeHook` in contract `BaseCCTPRelayer` (or the override in `NestCCTPRelayer`) so that, when hook validation fails, the message is marked in `retryHookQueue` (or a dedicated mapping) as refundable, allowing `refund` to succeed while optionally preventing further `retryHook` attempts; or.
- Introduce a separate state to flag messages with failed hooks and allow `refund` to rely on `nonceAmountReceived[nonce]` even when the hook was never queued.

This would keep funds recoverable via the standard user-facing flow, without depending on administrative `recoverToken` calls or losing per-nonce context.

Plume Network: Refactored the `relay` function to support forced refunds and updated error handling to not allow partial execution, removing unnecessary state variables from `BaseCCTPRelayer`. Fixed in commit 138d770a.

Cantina Managed: Fix verified.

3.2 Low Risk

3.2.1 Refund reuses untrusted finality threshold

Severity: Low Risk

Context: BaseCCTPRelayer.sol#L324

Description: Refunds burn back to the source chain using the `finalityThresholdExecuted` value embedded in the original message. That field is user-controlled. Using the attested value directly means that, in theory, we could refund using the fast transfer mode. The fast transfer mode is not supported by all chains, see cctp-supported-blockchains

Recommendation: When refunding, override `minFinalityThreshold` to 2000 so the standard-transfer mode is always used. This ensures refunds always use a safe confirmation depth regardless of the original message's setting and extra fees are not applied.

Plume Network: Fixed in commit 8bcce375.

Cantina Managed: Fix verified.

3.2.2 QuoteRelay lacks composer/selector validation

Severity: Low Risk

Context: NestCCTPRelayer.sol#L139-L156

Description: `quoteRelay` decodes hook data and calls `composer.quoteSend` without the allowlist/selector/length checks enforced during relay. A caller can supply malformed or untrusted `hookData` so that `quoteRelay` attempts to call an arbitrary composer/selector and reverts before producing a fee quote, or hits unintended external code paths. Relay itself validates, but the public quote path should match those guardrails to avoid surprises or DoS during the fee estimation:

```
// NestCCTPRelayer.quoteRelay
_validateMessage(_message);
(address target,,,, SendParam memory sendParam,) =
↪  _decodeNestHookData(_getHookData(_message, _data));
INestVaultComposer composer = INestVaultComposer(target);
return composer.quoteSend(address(this), composer.SHARE_OFT(), sendParam.amountLD,
↪  sendParam);
```

Recommendation: Apply the same hook validation used in `relay` before invoking the composer: check `hookData` length, composer allowlist, expected selector (`depositAndSend`) and amount vs received as appropriate. Reuse `_validateHookData` or duplicate its critical checks in `quoteRelay` to ensure only authorized composers/selectors are quoted.

Plume Network: Partially solved in commit 3cc143b3.

Cantina Managed: Acknowledged. The provided commit 3cc143b3 only adds the composer allowlist check and uses the decoded amount, but `quoteRelay` still skips the hook validation used in relay:

- In the `NestCCTPRelayer` the path calls `_validateMessage` then decodes hook data, checks `isComposer` and calls `composer.quoteSend(...)`. It does not run `_validateHookData`, so there is still no hook-length validity check and no selector check (`depositAndSend`), both of which relay enforces.
- `_decodeNestHookData` still accepts arbitrary selectors, but `quoteRelay` never filters them.

So the guardrails requested (selector/length parity with relay) remain missing; the issue is only partially addressed.

3.2.3 Missing validation for unmapped destination EIDs in `eidToDomain`

Severity: Low Risk

Context: NestCCTPRelayer.sol#L338

Description: The function `_depositForBurn` from contract `NestCCTPRelayer` sets the `destinationDomain` field using the expression `$.eidToDomain[_sendParam.dstEid]`. If `_sendParam.dstEid` has not been configured in storage, the mapping lookup returns the default value `0`, and the function proceeds using `destinationDomain = 0`.

Although the downstream `TokenMessengerV2` call will likely revert with an error such as "No TokenMessenger for domain", the current behaviour in `NestCCTPRelayer` does not perform any explicit validation on the existence of a mapping entry. This can make misconfiguration harder to diagnose and couples correctness to the behaviour of the external CCTP contract, instead of failing fast with a clear and protocol-specific error.

Recommendation: Consider to add an explicit check in the function `_depositForBurn` from contract `NestCCTPRelayer` to ensure that `eidToDomain[_sendParam.dstEid]` is non-zero (or otherwise marked as configured) before using it as `destinationDomain`, reverting with a dedicated error if the

mapping entry is missing. This would provide clearer feedback in case of configuration mistakes and reduce reliance on downstream revert reasons for correctness.

Plume Network: Fixed in commit f0573a26.

Cantina Managed: Fix verified.

3.2.4 Incorrect amount used in `quoteRelay` fee estimation

Severity: Low Risk

Context: NestCCTPRelayer.sol#L139-L156

Description: The function `quoteRelay` from contract `NestCCTPRelayer` decodes the hook data and calls:

```
composer.quoteSend(address(this), composer.SHARE_OFT(), sendParam.amountLD, sendParam);
```

to obtain the LayerZero fee quote.

In the Nest hook encoding, the `sendParam.amountLD` field is set to `0` by design, and the actual asset amount to be bridged is provided separately in the hook's `amount` value. As a result, `quoteRelay` is passing `sendParam.amountLD` (zero) as the `amountLD` argument when calling `composer.quoteSend`, instead of the real asset amount that will be used during execution.

This causes the returned `MessagingFee` to be underestimated (often zero), so callers may underpay the LayerZero fees. When the actual send is performed with a higher amount than the one used for quoting, the compose leg may fail or be refunded due to insufficient fees relative to the real bridged amount.

Recommendation: Consider to update the function `quoteRelay` from contract `NestCCTPRelayer` to use the actual asset amount decoded from the hook (the separate `amount` field in the hook encoding) when calling `composer.quoteSend`, instead of `sendParam.amountLD`. This will align the quoted fee with the real bridged amount and reduce the risk of compose failures or refunds due to underfunded LayerZero fees.

Plume Network: Fixed in commit e6b9d27.

Cantina Managed: Fix verified.

3.3 Informational

3.3.1 Fast transfer fee breaks CCTP deposit hooks

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The CCTP receiver assumes no fee is charged on mint: `_receiveMessage` records the net USDC minted, and `_validateHookData` requires that the hook's `assetAmount` (gross burn amount from the source message) is less than or equal to the received amount:

```
// BaseCCTPRelayer._receiveMessage
uint256 balanceBefore = USDC.balanceOf(address(this));
_relaySuccess = MESSAGE_TRANSMITTER.receiveMessage(_message, _attestation);
uint256 amountReceived = USDC.balanceOf(address(this)) - balanceBefore;
_getBaseCCTPRelayerStorage().nonceAmountReceived[_message.ref(0)._getNonce()] =
↪  amountReceived;
```

```
// NestCCTPRelayer._validateHookData
(address composer, bytes4 selector,, uint256 assetAmount,,) =
↪  _decodeNestHookData(_hookData);
if (assetAmount > _amountReceived) {
    return (true, abi.encodeWithSelector(Errors.InsufficientReceivedAmount.selector));
}
```

```
// BaseCCTPRelayer._executeHook
(bool invalid, bytes memory errorData) = _validateHookData(hookData,
↪  $.nonceAmountReceived[nonce]);
if (invalid) return (false, errorData);
```

When Circle executes a fast transfer and deducts a fee, the received amount becomes smaller than the encoded `assetAmount`, `_validateHookData` flags the hook as invalid and `_executeHook` returns without queuing a retry or refund. The net USDC then sits stranded on the relayer, while the user never receives vault shares nor a refund. Any non-zero `feeExecuted` or `maxFee` therefore halts deposits and traps funds on the destination.

Recommendation: Document this and ensure `depositForBurnWithHook` is always called by your backend and users with a `minFinalityThreshold` of 2000(Standard CCTP messages) and a 0 `maxFee`.

See for further reference the Circle CCTP technical guide.

Plume Network: Acknowledged. If the hook amount is set higher than the actual received amount, the hook call will fail. We need to account for that when building the deposit message. The funds will now be automatically refunded because: 138d770a.

Cantina Managed: Acknowledged.

3.3.2 Standard fee switch bricks burns/refunds

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: Both outbound burns and refunds hardcode `maxFee = 0`, while Circle's standard transfer fee switch requires `maxFee` to meet a minimum. If the switch is enabled, every burn with a hook will revert on the source chain and every refund burn will also revert, halting deposits and preventing recovery of queued funds. Even if a burn somehow succeeded with a fee applied, the destination would mint less than the encoded `assetAmount`, `_validateHookData` would reject the hook and the net USDC would remain stuck on the relayer without a retry/refund path. Circle's published matrix shows the switch is supported on Sei (other listed chains currently lack it), but the contracts should not assume it stays off everywhere.

```
// NestCCTPRelayer.send
CCTPDepositParams memory params = CCTPDepositParams({
    amount: amount,
    destinationDomain: $.eidToDomain[_sendParam.dstEid],
    mintRecipient: _sendParam.to,
    burnToken: address(USDC),
    destinationCaller: bytes32(0),
    maxFee: 0, // fails if standard fees are enforced
    minFinalityThreshold: $.finalityThreshold,
    hookData: ""
});
```

```
// BaseCCTPRelayer._refund
_depositForBurn(
    CCTPDepositParams({
        amount: $.nonceAmountReceived[nonce],
        destinationDomain: sourceDomain,
        mintRecipient: sender,
        burnToken: address(USDC),
        destinationCaller: bytes32(0),
        maxFee: 0, // refund burn also reverts when fees are required
        minFinalityThreshold: finalityThresholdExecuted,
        hookData: new bytes(0)
    })
);
```

```
// TokenMessengerV2._depositForBurn
require(_maxFee >= _calcMinFeeAmount(_amount), "Insufficient max fee");
```

Recommendation: Make the relayer fee-aware: supply a configurable `maxFee` that satisfies the standard fee requirement for burns and refunds, validate against Circle's minimums and pass the net minted amount through hook validation and composer calls.

Plume Network: Fixed in commit c2742dbe.

Cantina Managed: Fix verified.

3.3.3 Open `destinationCaller` lets others consume CCTP message

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: Burn messages set `destinationCaller` to `bytes32(0)`, allowing anyone to invoke `MessageTransmitter.receiveMessage` once the attestation is public. An arbitrary caller can pre-consume the CCTP message: USDC mints to the relayer, but the authorized `relay` call later reverts because `_receiveMessage` sees the message already executed and never writes nonce bookkeeping. The hook is never executed, so no vault shares are minted or forwarded and the minted USDC remains stranded on the relayer.

Recommendation: Document this and ensure `depositForBurnWithHook` is always called with the `destinationCaller` parameter set to the destination relayer address so only the relayer can consume the message.

Plume Network: Acknowledged. Proper documentation on how to interact with Nest OVaults through CCTP will be added to the public repo.

Cantina Managed: Acknowledged.

3.3.4 Unused import

Severity: Informational

Context: NestCCTPRelayer.sol#L6

Description: The `NestCCTPRelayer` contract imports `NestVaultComposer` but never uses this type.

```
import {NestVaultComposer} from "contracts/ovault/NestVaultComposer.sol";
```

Recommendation: Remove the unused import to keep the relayer source minimal.

Plume Network: Fixed in commit edc1d578.

Cantina Managed: Fix verified.

3.3.5 Payable `send` function does not enforce `msg.value == 0`

Severity: Informational

Context: BaseCCTPRelayer.sol#L308-L327

Description: The function `send` from contract `NestCCTPRelayer` is declared as `payable` but does not make any use of `msg.value`. The function does not forward native tokens, does not consume them as fees, and does not refund them to the caller. As a result, any ether sent along with the call is kept in the contract balance.

In the current design, the `NestCCTPRelayer` is used as an `IOFT` implementation where cross-chain transfers are performed via CCTP and do not require LayerZero native fees, so callers are expected to invoke `send` with `msg.value = 0`. However, the function signature still accepts non-zero `msg.value` without reverting, which can lead to accidental funding of the contract and requires the use of privileged recovery mechanisms (for example, `recoverToken(address(0), ...)`) to retrieve the locked native tokens.

Recommendation: Consider to enforce `msg.value == 0` in the function `send` from contract `NestCCTPRelayer` (for example, by reverting when `msg.value > 0`) to prevent accidental locking of native tokens and make the intended usage explicit. If accepting native tokens is not part of the design,

consider to remove the `payable` modifier or document clearly that `msg.value` must always be zero and that any non-zero value will be treated as an error.

Plume Network: Added documentation in commit acf9f46f. Funds are recoverable through the `recoverTokens` function.

Cantina Managed: Fix verified.